

# 从 Vibe Coding 到 Harness Engineering，再到 Trellis 落地

今天想和大家聊三个关键词：

- **Vibe Coding**
- **Harness Engineering**
- **Trellis**

它们看起来像是三个不同层次的话题，但其实可以串成一条很清晰的主线：

**Vibe Coding** 负责推进开发流程，**Harness Engineering** 负责让流程稳定可控，**Trellis** 负责把这套能力工程化落地。

## 一、Vibe Coding 工作范式：从需求到验证的闭环

先说第一个关键词：**Vibe Coding**。

**不是一上来就追求完美，而是先把想法推进到可验证状态。**

也就是说，Vibe Coding 的核心不是“写得快”，而是“尽快进入一个可以讨论、可以验证、可以迭代的状态”。

### 1. 原型图设计

原型图的意义：

- 让需求方更直观地理解流程
- 让研发提前看到交互路径
- 让所有人对同一件事形成共同认知

很多时候，原型图比长篇文字更能暴露问题。

因为一旦流程画出来，哪里不合理、哪里太复杂、哪里漏了状态，马上就会显现出来。

所以原型图这一层，本质上是在做一件事：

**把抽象需求变成可视化的讨论对象。**

### 2. 需求澄清

第一步一定不是写代码，而是澄清需求。

很多问题不是开发慢，而是一开始就没有把目标讲清楚。

如果目标不清楚，后面 PRD、原型、前端、后端、测试都会跟着偏。

这一步的价值在于：  
把模糊的诉求，转成可执行的问题定义。

### 3. PRD 输出

需求澄清之后，就要把口头描述沉淀成 PRD 或 Spec。

这一层的重点不是“写得漂亮”，而是“写得清楚”。

这一步其实是在回答一个很关键的问题：

**“这个东西应该长什么样，做到什么程度，才算完成？”**

如果没有这一步，后面 AI 很容易凭感觉发挥，最后生成一堆看起来像样、但并不真正对齐需求的内容。

### 4. 前端实现

原型图+PRD确认后，前端开始落地。

这一阶段的重点不只是“页面画出来”，而是要确保：

- 页面结构和 PRD 一致
- 交互逻辑和原型一致
- 状态流转没有遗漏
- 关键异常场景能够处理

### 5. 后端实现

前端有了，后端就要跟上。

后端这层最重要的不是“有接口”，而是：

- 接口定义是否清晰
- 数据结构是否合理
- 业务规则是否完整
- 权限、异常、边界是否处理到位

如果说前端负责“让用户看到并操作”，后端负责的就是“让系统真正正确地跑起来”。

### 6. 测试与评测

最后一步不是结束，而是验证。

无论是单测、集成测试，还是自动化评测，核心目的都不是“证明我写了”，而是“证明它真的对了”。

测试的价值在于把“看起来能用”变成“稳定能用”。

而更关键的一点是：

评测结果不是终点，而是反向推动规格和实现迭代的输入。

这意味着 Vibe Coding 不是线性的，而是一个闭环：

需求澄清 → PRD → 原型图 → 前端 → 后端 → 测试 → 反向迭代

如果说传统开发更像“写完再验证”，那 Vibe Coding 更像“边推进边反馈”。

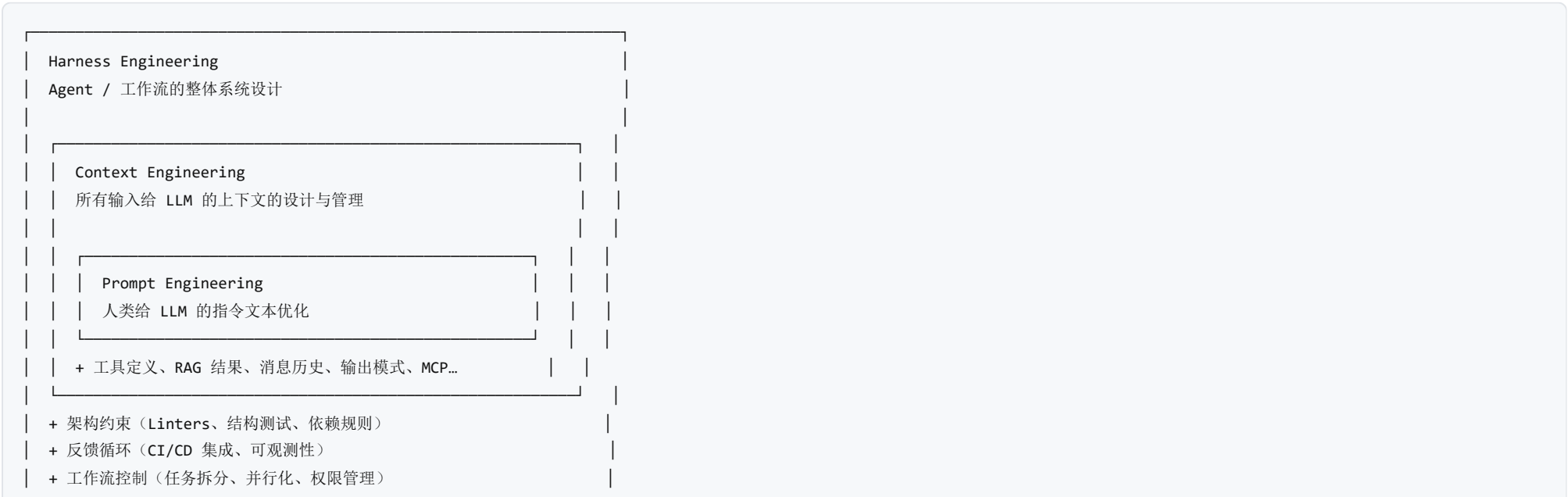
## 二、Harness Engineering：让 AI 写代码从“会写”走向“能用”

如果说 Vibe Coding 解决的是“怎么把开发流程推进起来”，那 **Harness Engineering** 解决的就是另一个更深的问题：

怎么让 AI 在真实项目里持续稳定地做对事。

### 1. AI 写代码的三种境界

2024-2026 年，AI 工程领域依次出现了三个核心概念。它们不是替代关系，而是 **层层嵌套**：



| + 改进周期（熵管理、自动清理、文档保鲜）|

第一层：Prompt Engineering — 怎么写指令

大家最熟悉的。写好提示词，让模型输出更准。few-shot、chain-of-thought、角色设定都属于这一层。

第二层：Context Engineering — 给模型看什么

光写好 prompt 不够，还要设计好模型的完整输入：工具定义、RAG 检索结果、消息历史、结构化输出模式。很多人已经在做这件事了。

第三层：Harness Engineering — 防止什么、控制什么、修复什么

这是今天的重点。前两层解决的是“单次推理怎么更好”，但当 Agent 跑在真实项目里、跨多个会话、持续产出代码时，你需要的是 **整个系统的持续质量**。

| 维度   | Prompt Engineering | Context Engineering | Harness Engineering     |
|------|--------------------|---------------------|-------------------------|
| 核心问句 | “怎么写指令”            | “给模型看什么”            | “防止什么、控制什么、修复什么”        |
| 失败模式 | 回答跑偏               | 单次输出不准确             | 质量随时间退化                 |
| 典型实现 | 提示词优化              | RAG、Memory、工具定义     | Linters、CI 集成、任务拆分、自动清理 |

Harness 这个词本来是“马具”的意思。  
马本身很有力量，但你不能只靠它乱跑；你需要缰绳、马鞍、嚼子，把力量导向正确方向。

放到 AI 里也一样：

- **Model** 是能力本体
- **Harness** 是让能力可控、可验证、可持续的系统

所以可以非常直接地理解成：

Agent = Model + Harness

模型负责“想”，Harness 负责“管”。

如果没有 Harness，模型可能很强，但它：

- 不一定遵守规范
- 不一定记得进度
- 不一定知道自己哪里错了
- 不一定能在长任务里保持一致性

### 3.1 几种 Harness 开源方案

当前个人开发者可选的 harness 工具和方案大致可分为三种：

**纯配置方案。** 最轻量的起点是直接编写 CLAUDE.md / AGENTS.md，配合 superpowers 等 skill 集合。这一方案的核心是把配置层和约束层的实践手动落地。

**框架增强方案。** Trellis 和 ccg-workflow 等框架在配置层之上增加了 spec 系统和 hook 自动注入能力。以 Trellis 为例，它通过三个 hook 时机（session-start、inject-subagent-context、ralph-loop）确保 spec 文档在每次对话中都被使用，而不是写完就被遗忘。Trellis 选择将沉淀层系统化：spec 通过 hook 自动注入到每次交互中的强制上下文。

**多角色编排方案。** oh-my-claudecode、oh-my-openencode 等套件定义了多种 agent 角色（planner、executor、reviewer、debugger 等），实现了角色分工和多 agent 协作。

在个人实践中，通常做好配置层（写好 CLAUDE.md，配好 lint，让 Agent 清楚规矩）即可。对于持续迭代的项目，通常可以涉及配置层 + 约束层 + 适度的 spec 沉淀即可。对于长期项目，可以考虑 Trellis 等增强框架。

然而，值得注意的是，Harness 说到底是在模型智能的基础上构建系统，随着模型基础能力的提升，Harness 的部分组件应该能够简化甚至移除。

Anthropic 的研究对此的原话是：

every component in a harness encodes an assumption about what the model can't do on its own, and those assumptions are worth stress testing, both because they may be incorrect, and because they can quickly go stale as models improve.

Harness 的每一个组件都编码了对模型自身能力的某种假设，这些假设值得反复验证，因为它们可能本来就是错的，也可能很快会因模型进步而过时。

一个可供参考的判断方法：**如果在项目中花了大量时间在维护 Harness 本身（编辑计划文件、更新进度文档、协调 Agent 之间的通信）而实际代码产出很少，说明 Harness 已经变成了负担而非助力。**

## 三、Trellis 框架：把 Harness Engineering 工程化落地

前面讲了 Vibe Coding 和 Harness Engineering，接下来就是第三部分：**Trellis**。

我对 Trellis 的理解是：

**Trellis 是把 Harness Engineering 真正工程化、自动化、可持续化的一套框架。**

它不是一个单纯的提示词工具，也不是一个简单的技能集合，而是一整套把规范、任务、上下文、验证、沉淀串起来的系统。

### 1. Trellis 和其他方案的对比

如果和 Superpowers、多角色编排方案 这类方案放在一起看，可以这样理解：

#### 1.1 Superpowers

## How it works

它从你启动 coding agent 的那一刻开始工作。一旦它发现你正在构建某个东西，它不会立刻冲进去写代码，而是先退一步，问你真正想达成的目标是什么。

当它从对话中整理出一份 spec 之后，会把它拆成足够短的小块展示给你，让你真的能读完、消化掉。

在你确认设计之后，agent 会整理出一份实现计划。这个计划清晰到就算交给一个热情但品味一般、判断力不足、没有项目上下文、还不爱测试的初级工程师，也能跟着做。它强调真正的 red/green TDD、YAGNI (You Aren't Gonna Need It) 和 DRY。

接下来，一旦你说“开始”，它就会启动一个 *subagent-driven-development* 流程，让多个 subagent 逐个处理工程任务，检查并评审彼此的工作，然后继续推进。Claude 连续自主工作几个小时而不偏离你制定的计划，这并不罕见。

除此之外还有很多内容，但这就是系统的核心。并且因为这些 skills 会自动触发，你不需要做任何额外操作。你的 coding agent 天生就拥有 Superpowers。

### 1.2 多角色编排方案

以 oh-my-openagent 举例：

#### 1.2.1 Sisyphus: The Discipline Agent

西西弗斯：纪律型 Agent

Sisyphus 是你的主编排器。他负责规划、把任务委派给专门的子 agent，并通过高强度并行执行把任务推进到完成。他不会半途而废，不会分心，他会把事情做完。

**推荐模型：**

**Claude Opus 4.6** — 综合体验最佳。Sisyphus 使用了针对 Claude 优化的提示词。

**Claude Sonnet 4.6** — 能力与成本之间的平衡很好。

**Kimi K2.5** — 很棒的类 Claude 替代方案。很多用户只用这一套组合。

**GLM 5** — 一个稳妥的选择，尤其适合通过 Z.ai 使用。

#### 1.2.2 Hephaestus: The Legitimate Craftsman

赫菲斯托斯：名正言顺的工匠

这个名字带有刻意的反讽意味。由于这个项目，Anthropic 阻止了 OpenCode 使用他们的 API。所以团队转而构建了一个原生于 GPT 的自治 agent。

Hephaestus 运行在 GPT-5.4 上。给他一个目标，而不是一份步骤清单。他会探索代码库、研究模式，并端到端执行任务，无需手把手指导。他之所以是“名正言顺的工匠”，是因为他诞生于必要，而不是特权。

当你需要深入的架构推理、跨多个文件的复杂调试，或跨领域知识整合时，就使用 Hephaestus。当工作明确需要 GPT-5.4 的特定优势时，也应主动切换到它。

### Why this beats vanilla Codex CLI:

**这为什么比原生 Codex CLI 更强：**

**多模型编排。** 纯 Codex 是单模型。OmO 会自动把不同任务路由给不同模型：GPT 负责深度推理，Gemini 负责前端，GPT-5.4 Mini 负责速度。把合适的大脑用在合适的工作上。

**后台 agents。** 可以并行启动 5 个甚至更多 agent。这是 Codex 根本做不到的。一个 agent 写代码，另一个研究模式，再另一个查文档，就像一个真实的开发团队。

**分类系统。** 任务按意图路由，而不是按模型名。visual-engineering 分配给 Gemini，ultrabrain 分配给 GPT-5.4，quick 分配给 GPT-5.4 Mini。不需要手动切换。

**积累式智慧。** 子 agent 会从之前的结果中学习。第 1 个任务里发现的约定，会传递给第 5 个任务。早期犯过的错误不会再重复。系统会随着工作推进而变得更聪明。

### 1.2.3 Prometheus: The Strategic Planner

普罗米修斯：战略规划师

Prometheus 会像真实工程师一样“面试”你：提出澄清问题、识别范围与歧义，并在动任何代码之前先制定详细计划。

按 **Tab** 进入 Prometheus 模式，或者在 Sisyphus 中输入 @plan "你的任务"。

## 1.3 Trellis

**Trellis 是 AI 编码助手的脚手架** — 通过自动化机制注入项目规范，让 AI 每次都按你的标准写代码，而不是随机发挥。

所以这三者的区别可以简单概括为：

- **Superpowers**：技能包
- **oh-my-openagent**：多角色编排
- **Trellis**：AI 编码脚手架

## 2. 为什么选择 Claude Code + Trellis

Trellis 之所以能真正发挥作用，很大程度上是因为它和 Claude Code 的 hook 机制天然契合。

### 2.1 Claude Code 众多生命周期之一：工具调用生命周期

工具调用生命周期是钩子系统中使用频率最高、功能最强大的一组事件。它们构成了一个“三明治”结构：PreToolUse 在执行前拦截，PostToolUse 在成功后处理，PostToolUseFailure 在失败后兜底。

**PreToolUse** （工具执行前）

**PostToolUse** （工具执行后）

**PostToolUseFailure** （工具执行失败后）

### 2.2 Trellis Hook 执行顺序

#### 2.2.1 session-start：会话启动时注入上下文

| 读取内容                      | 来源      | 用途          |
|---------------------------|---------|-------------|
| .trellis/.developer       | 开发者身份   | 知道"你是谁"     |
| .trellis/workflow.md      | 工作流指南   | 知道"怎么工作"    |
| workspace/{name}/index.md | 会话历史    | 知道"上次做了什么"  |
| git log                   | Git提交记录 | 知道"最近改了什么"  |
| .trellis/tasks/           | 活跃任务列表  | 知道"当前任务是什么" |

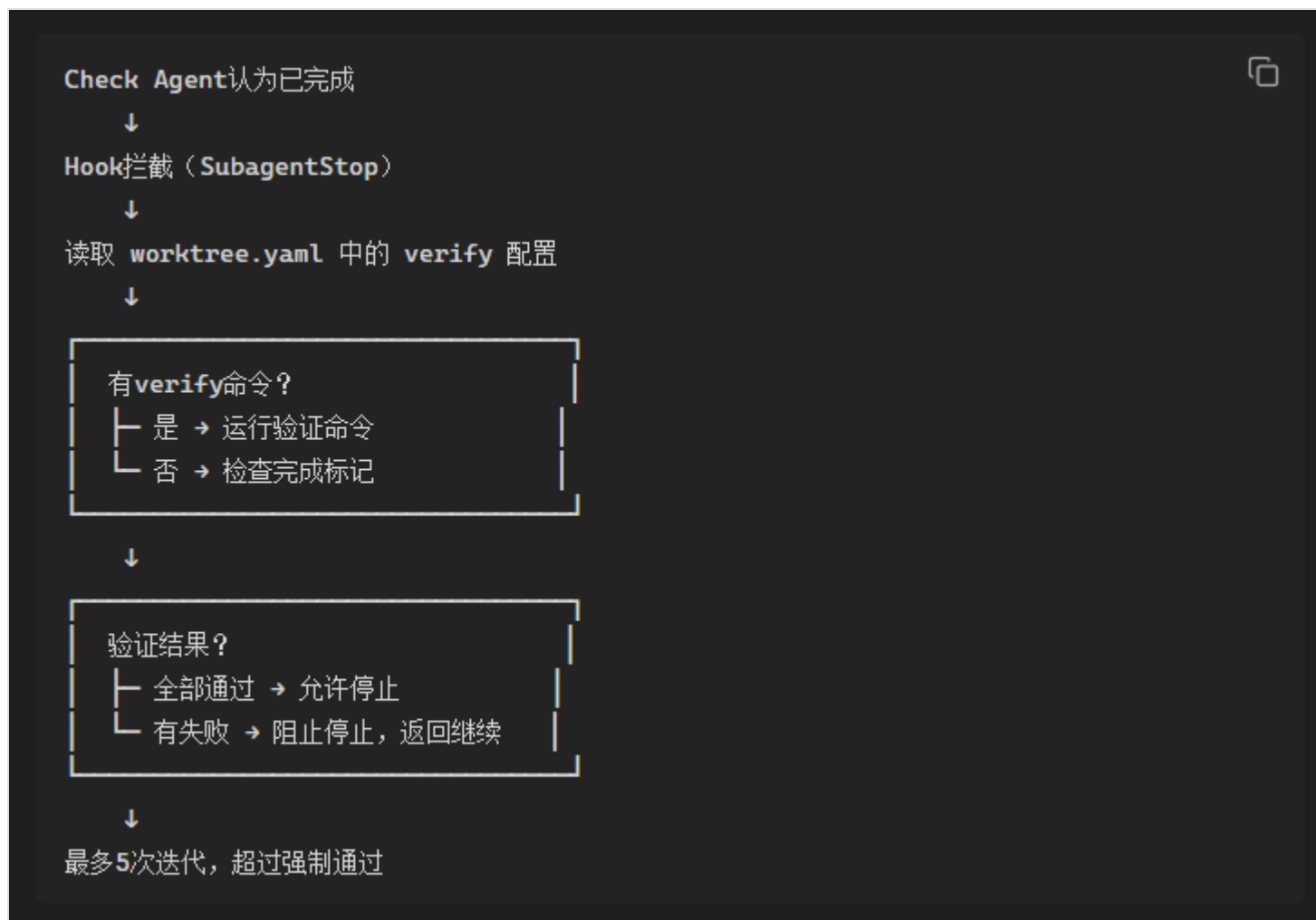
2.2.2 inject-subagent-context: Agent调用时注入精确Spec。

| Agent          | 注入内容                                  | 触发条件      |
|----------------|---------------------------------------|-----------|
| implement      | implement.jsonl + prd.md + info.md    | 实现功能时     |
| check          | check.jsonl + prd.md                  | 代码检查时     |
| check [finish] | finish-work.md + prd.md               | 完成检查时（轻量） |
| debug          | debug.jsonl + codex-review-output.txt | 调试bug时    |
| research       | 项目结构概览 + research.jsonl               | 信息搜索时     |

2.2.3 ralph-loop: Check完成后自动质量验证

|                |
|----------------|
| Check完成后自动质量验证 |
|----------------|





### 3. Trellis 的日常使用场景

#### 3.1 首次使用(AI自动扫描项目生成spec文档)

```
npm install -g @mindfoldhq/trellis@latest
```

```
cd your-project
```

```
trellis init --claude -u your-name
```

```
/trellis:start
```

### 3.2 日常使用三步走

```
/trellis:start
```

```
-> 输入需求...
```

```
-> vibe coding...
```

```
/finish-work #检查完工清单
```

```
/record-session #会话持久化存储
```

### 3.3 Bug修复, 以及SPEC规范沉淀

```
-> 修复bug后
```

```
# 分析bug原因
```

```
/break-loop
```

```
-> 分析完成, 告诉ai, 将bug原因记录到.trellis/spec/ 文件
```

## 4. Trellis 的 spec 产物结构

Trellis 的另一个重点, 是它把规范沉淀成了明确的目录结构。

一个典型结构大致可以理解成这样:

```
.trellis/spec/
├─ frontend/           # 前端规范
│   ├─ index.md        # 索引: 列出所有规范及状态
│   ├─ component-guidelines.md # 组件规范
│   ├─ hook-guidelines.md # Hook 规范
│   ├─ state-management.md # 状态管理
│   ├─ type-safety.md   # 类型安全
│   ├─ quality-guidelines.md # 质量指南
│   └─ directory-structure.md # 目录结构
```

```
|
|— backend/                # 后端规范
|   |— index.md            # 索引：列出所有规范及状态
|   |— database-guidelines.md # 数据库规范
|   |— error-handling.md   # 错误处理
|   |— logging-guidelines.md # 日志规范
|   |— quality-guidelines.md # 安全质量规范
|   └— directory-structure.md # 目录结构
|
└— guides/                 # 思维指南
    |— index.md
    |— cross-layer-thinking-guide.md # 跨层思维指南
    └— code-reuse-thinking-guide.md # 代码复用指南
```

## 5. 团队协作

```
# 加入新成员
/trellis-onboard
```

**不冲突的部分**（各人独立）：

- `workspace/{name}/` — 每人独立目录
- `.developer` — gitignored, 每人独立
- 并行任务的 `worktree` — 物理隔离

**可能冲突的部分**（需要协调）：

- `spec/` — 多人可能同时修改同一规范
- `tasks/` — 多人可能操作同一任务

**最佳实践：**

- Spec 修改通过 PR 审查
- 任务指派明确（`--assignee` 参数）
- 重要的 spec 变更在团队会议上讨论

## 6. 其他一些Trellis命令

| 功能                   | Claude Code / OpenCode                         |
|----------------------|------------------------------------------------|
| 启动会话                 | <code>/start</code> （可选，Hook 自动注入）             |
| 后端准备                 | <code>/before-backend-dev</code> （可选）          |
| 前端准备                 | <code>/before-frontend-dev</code> （可选）         |
| 检查后端                 | <code>/check-backend</code> （Ralph Loop 自动触发）  |
| 检查前端                 | <code>/check-frontend</code> （Ralph Loop 自动触发） |
| 跨层检查                 | <code>/check-cross-layer</code>                |
| 完工清单                 | <code>/finish-work</code>                      |
| 并行编排                 | <code>/parallel</code>                         |
| 记录会话                 | <code>/record-session</code>                   |
| 头脑风暴                 | <code>/brainstorm</code>                       |
| Bug 分析               | <code>/break-loop</code>                       |
| 新成员入门                | <code>/onboard</code>                          |
| 创建命令                 | <code>/create-command</code>                   |
| 集成 Skill             | <code>/integrate-skill</code>                  |
| ### 7. Trellis 的核心价值 |                                                |

我觉得 Trellis 的价值不在于 “又多了一套工具”，而在于它把以下几件事真正连起来了：

- 规范
- 任务
- 上下文
- 验证
- 历史沉淀

所以它不是让 AI 变得更神奇，而是让 AI 的工作方式更像一个真正可运行的工程系统。

---

## 四、把三者串起来：从“会用 AI”到“驾驭 AI”

现在我们把三部分连起来看，主线就会非常清楚。

### 1. Vibe Coding 负责起步

先把需求推进到可讨论、可验证、可迭代的状态。

### 2. Harness Engineering 负责稳定

让 AI 在真实项目里持续正确地工作，而不是今天对、明天错。

### 3. Trellis 负责落地

把方法变成系统，把经验变成规范，把规则变成自动化机制。

如果用一句话总结今天的内容，我会说：

真正优秀的 AI Coding，不是让模型替你写完代码，而是让你有能力设计一套系统，使模型持续稳定地把事情做对。

## 五、结语

最后做一个简短总结。

今天我们其实讲了三层东西：

- **Vibe Coding**：怎么把开发流程快速推进起来
- **Harness Engineering**：怎么让 AI 编码长期稳定可控
- **Trellis**：怎么把这套能力工程化落地

如果只会 Vibe Coding，可能会快，但不一定稳。

如果只有 Harness 的概念，没有落地系统，可能知道该怎么做，但不一定能持续用。

如果有 Trellis 这样的框架，才能把方法变成可持续的工程能力。

所以我对这场分享最想留下的一句话是：

AI 编码的重点，已经不只是“会不会写”，而是“能不能被驾驭”。